Zewail City of Science, Technology and Innovation

University of Science and Technology

School of Computational Sciences and Artificial Intelligence

CSAI 203 - Fall 2025

Introduction to Software Engineering

Software Requirements Specification (SRS)

Project: Clinexa Medical System

Version 1.0

Team Number: 26

Email: s-ahmed.ragab@zewailcity.edu.eg

Team Members

Name	**ID**	**Email**
Ahmed Ragab	202400437	s-ahmed.ragab@zewailcity.edu.eg
Mosab Kamal	202402297	s-mosab.gabal@zewailcity.edu.eg
Mohamed Maher	202301798	s-mohammed.kafagy@zewailcity.edu.eg
Abdelrhman Khaled	202400161	s-abdelrhman.abdelhamid@zewailcity.edu.eg

Contents

1 Introduction **3**

1.1 Purpose . . . 3

1.2 Scope . . . 3

1.3 Definitions . . . 3

1.4 References . . . 4

1.5 Overview . . . 4

2 System Overview **4**

2.1 Platform Perspective . . . 4

2.2 Application Functions . . . 4

2.3 User Classes . . . 5

2.4 Operating Environment . . . 5

2.5 Constraints . . . 5

2.6 User Documentation . . . 5

2.7 Assumptions and Dependencies . . . 6

3 Specific Requirements **6**

3.1 Functional Requirements . . . 6

3.1.1 FR1: User Login . . . 6

3.1.2 FR2: Edit Profile . . . 6

3.1.3 FR3: Add / Remove Patient . . . 6

3.1.4 FR4: Schedule Appointment . . . 7

3.1.5 FR5: Manage Appointments . . . 7

3.1.6 FR6: Search . . . 7

3.1.7 FR7: Generate Reports . . . 7

3.1.8 FR8: Send Notifications . . . 7

3.1.9 FR9: Manage Medical Records . . . 8

3.1.10 FR10: Feedback and Rating System . . . 8

3.2 Use Case Model . . . 8

3.2.1 Use Case Diagram . . . 8

3.2.2 Use Case Descriptions 9
3.3 Domain Model 10
3.3.1 Conceptual Class Diagram 10
3.3.2 Class Descriptions 10
3.4 Non-Functional Requirements 12
3.4.1 Reliability and Availability Requirements 12
3.4.2 Maintainability 12
3.5 External Interface Requirements 13
3.5.1 User Interface 13
3.5.2 Hardware Interface 13
3.5.3 Software Interface 13
3.5.4 Communication Interface 13

4 Appendices **14**
4.1 Appendix A: Data Dictionary 14
4.2 Appendix B: Glossary 14

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for Clinexa, a smart healthcare platform that facilitates communication and management between doctors and patients. This document provides a detailed description of the system's features, user interactions, and operational constraints. It serves as a reference for the development team, stakeholders, and testers throughout the software development life cycle

1.2 Scope

Clinexa is a web-based healthcare management system designed to improve the efficiency and accessibility of medical services. The platform allows patients to book appointments, communicate with doctors, and access their medical records. Doctors can also manage patient data and schedule consultations.

The system aims to:

- Enhance communication between doctors and patients.
- Simplify appointment scheduling and medical record management.
- Reduce administrative overhead in healthcare services.

1.3 Definitions

- **SRS**: Software Requirements Specification.
- **UI**: User Interface.
- **UX**: User Experience.
- **CMS**: Clinexa Management System.
- **Clinexa**: The proposed smart healthcare management system.
- **Patient**: The end user who uses the system to book appointments and access healthcare services.

- **Doctor**: The medical professional who provides consultations and manages patient records within the system.

1.4 References

1. Course Project Guidelines – CSAI 203 Introduction to Software Engineering, Fall 2025.
2. Project Repository Documentation (Clinexa - GitHub).

1.5 Overview

This document outlines the purpose, scope, and functionality of the Clinexa system. Subsequent sections will describe:

- System features and user requirements.
- Functional and non-functional requirements.

The goal is to ensure that all stakeholders have a clear, shared understanding of what the system will deliver.

2 System Overview

2.1 Platform Perspective

The CMS replaces manual paper-based systems with a digital solution (web application) to reduce errors, save time, and improve coordination between doctors and patients.

2.2 Application Functions

- User login and role-based access
- Manage doctor and patient profiles
- Schedule, reschedule, cancel appointments
- Search patients and appointments

- Generate operational reports
- Send automated notifications
- Link patients to their doctors

2.3 User Classes

Different types of users who will interact with the system:

- **Registered Users**: Can browse time slots, book anyone.
- **Doctors**: Can specify time slots, cancel appointments, and show patient medical history

2.4 Operating Environment

The system will operate in the following environments:

- **Client-side**: Runs in web browsers such as Chrome and Firefox.
- **Server Side**: Hosted on a cloud server running Flask.
- **File system**: Storing and reading data from the file.

2.5 Constraints

- Must use UML 2.5 for diagrams.
- Backend: Flask.
- Frontend: (HTML and CSS).
- Diagrams: draw.io or Lucidchart.

2.6 User Documentation

- User Manual (PDF)
- Online Help within the system

2.7 Assumptions and Dependencies

- Internet connection available
- Users have basic computer skills
- Email/SMS gateway API available

3 Specific Requirements

3.1 Functional Requirements

3.1.1 FR1: User Login

Scenarios:

- Valid credentials → access dashboard (all features)
- Invalid → show error (3 attempts → lock 5 mins)
- Forgotten password → reset via email/receptionist

3.1.2 FR2: Edit Profile

Scenarios:

- Doctor updates availability hours
- Receptionist updates contact info

3.1.3 FR3: Add / Remove Patient

Scenarios:

- Add a patient with a medical history
- Remove patient (archive, not delete from DB)

3.1.4 FR4: Schedule Appointment

Scenarios:

- Check doctor availability
- Prevent double-booking
- Cancel the booked appointment if the patient wants

3.1.5 FR5: Manage Appointments

Scenarios:

- The doctor views today's list
- Patient cancels 24h before

3.1.6 FR6: Search

Scenarios:

- Partial name match
- Search appointments by date/doctor

3.1.7 FR7: Generate Reports

Scenarios:

- Daily appointments
- Patient visit history
- Doctor workload

3.1.8 FR8: Send Notifications

Scenarios:

- 24h before appointment → SMS/Email
- Appointment confirmed → both parties

3.1.9 FR9: Manage Medical Records

Scenarios:

- Add a new prescription after consultation.
- View patient history before the new appointment.
- Update diagnosis or notes.

3.1.10 FR10: Feedback and Rating System

Scenarios:

- The patient submits feedback after the completed session.
- Admin reviews ratings for performance analysis.

3.2 Use Case Model

3.2.1 Use Case Diagram

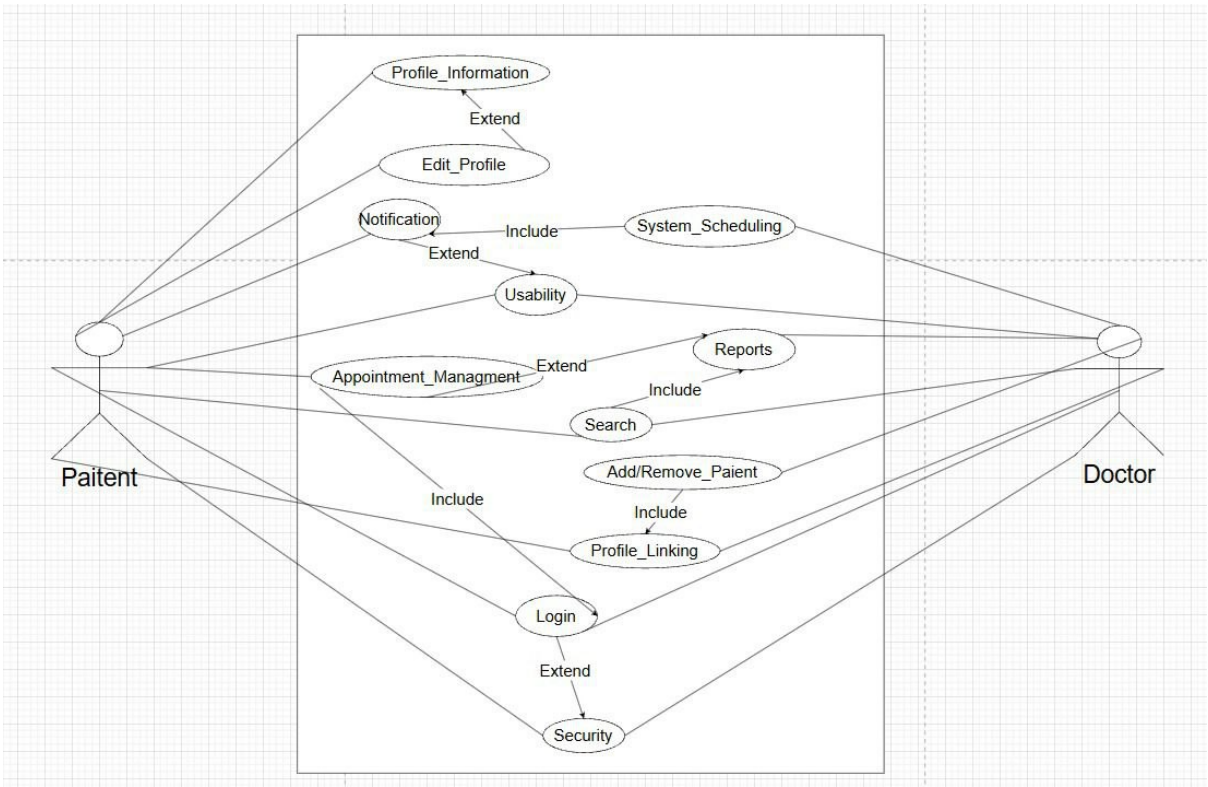


Figure 1: Use Case Diagram

3.2.2 Use Case Descriptions

Use Case	Actors	Description	Relationships
Login	Patient, Doctor	Allows users to access the system securely.	extend Security
Edit_Profile	Patient	Enables users to update their personal information.	extend Profile_Information
Profile_Information	Patient	Displays user profile details.	
Notification	Patient	Sends alerts or updates to users.	extend Usability
System_Scheduling	Doctor	Manages scheduling of appointments and availability.	include Notification
Appointment Management	Patient	Handles booking, rescheduling, and cancellation of appointments	extend Reports include Login
Reports	Doctor	Generates and views patient reports.	
Search	Patient, Doctor	Allows users to search for information or profiles.	include Reports
Add/Remove_Patient	Doctor	Enables doctors to manage patient records.	include Profile_Linking
Profile_Linking	Doctor	Links patient profiles for shared access or referrals.	
Usability	Patient, Doctor	Enhances user experience and accessibility.	

Use Case	Actors	Description	Relationships
Security	Patient, Doctor	Ensures data protection and secure access.	

3.3 Domain Model

3.3.1 Conceptual Class Diagram

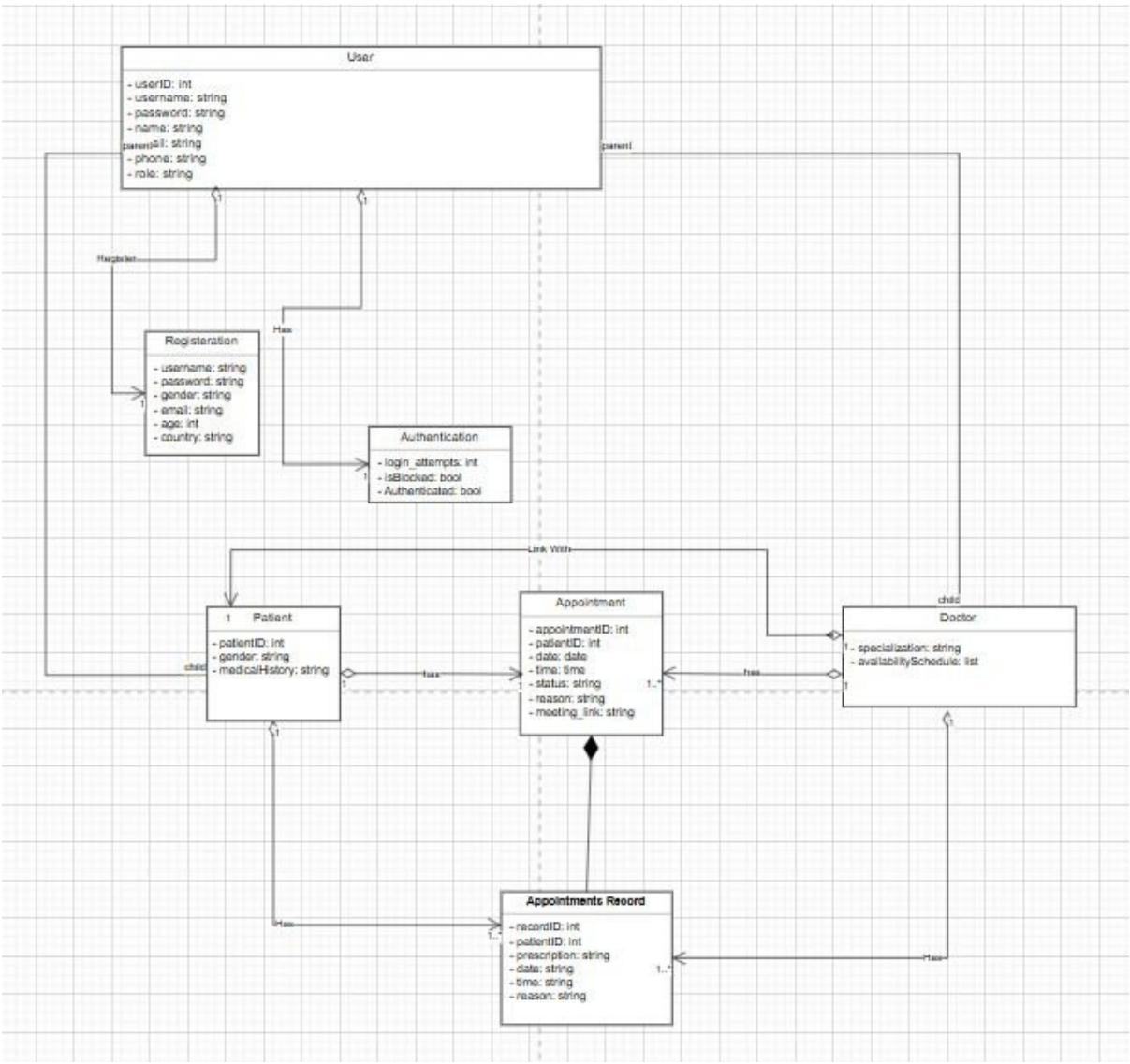


Figure 2: Conceptual Class Diagram

3.3.2 Class Descriptions

Class name	**Description**	**Key Attributes (with Data type)**	**Associations / Relationships**
User	Base class representing any system user (doctor, patient, admin).	userID: int, username: string, password: string, name: string, email: string, phone: string, role: string	Inherited by Doctor and Patient; associated with Authentication (login) and Registration (sign up).
Authentication	Handles user login attempts and authentication state.	login_attempts: int, isBlocked: bool, Authenticated: bool	Associated with User (1-to-1); manages login control and validation.
Registration	Manages new user registration and data collection.	username: string, password: string, gender: string, email: string, age: int, country: string	Linked to User (for creating accounts); used by Patient during sign-up
Patient	Represents a patient user with medical information	patientID: int, gender: string, medicalHistory: string	Inherits from User; associated with Appointment, Appointment Record, and Registration

Class name	**Description**	**Key Attributes (with Data type)**	**Associations / Relationships**
Doctor	Represents a doctor with specialization and working hours	specialization: string, availabilitySchedule: list	Inherits from User; associated with Appointment (for booking) and Appointment Record (for viewing history).
Appointment	Represents a booked appointment between patient and doctor.	appointmentID: int, patientID: int, date: date, time: time, status: string, reason: string, meeting_link: string	Linked to Patient and Doctor (many-to-one each); connected to Appointment Record.
Appointments Record	Stores consultation details and prescription data	recordID: int, patientID: int, prescription: string, date: string, time: string, reason: string	Linked to Appointment (composition) and Doctor (association for access).

3.4 Non-Functional Requirements

3.4.1 Reliability and Availability Requirements

- The platform should maintain 99.9% uptime.
- Recovery time after a system failure should be within 5 minutes.

3.4.2 Maintainability

- The success rate of the maintenance will be 90%
- The maintenance speed will be 95%.

3.5 External Interface Requirements

3.5.1 User Interface

Clinexa will provide a responsive web-based interface accessible from any browser (Google Chrome, Firefox).

The interface will include login pages, dashboards for doctors and patients, appointment scheduling screens, and show reports

Patients will have access to booking and show reports, while doctors will have access to patient lists, schedules and reports.

3.5.2 Hardware Interface

The system will run on standard computing devices, such as desktops, laptops, and tablets.

No special hardware is required, except a device with a stable internet connection and web browser.

3.5.3 Software Interface

The system will run on standard computing devices, such as desktops, laptops, and tablets.

No special hardware is required, except a device with a stable internet connection and web browser.

- **Operating System**: Any OS capable of running modern web browsers (Windows, macOS, Linux).
- **Backend Framework**: Flask.
- **Frontend**: HTML, CSS.
- **Database**: File System

3.5.4 Communication Interface

Clinexa will use HTTP/HTTPS protocols for client–server communication to ensure security and reliability.

SMTP protocol will be used for email notifications.

4 Appendices

4.1 Appendix A: Data Dictionary

- **SRS**: Software Requirements Specification.
- **UI**: User Interface.
- **UX**: User Experience.
- **CMS**: Clinexa Management System.
- **Clinexa**: The proposed smart healthcare management system.
- **Patient**: The end user who uses the system to book appointments and access healthcare services.
- **Doctor**: The medical professional who provides consultations and manages patient records within the system.

4.2 Appendix B: Glossary

- **Clinexa**: The smart healthcare management system developed in this project
- **User**: Any individual interacting with the system (Doctor, Patient, or Admin)
- **Doctor**: A medical professional responsible for consultations and managing patients.
- **Patient**: A registered user who books appointments and receives medical care
- **Appointment**: A scheduled meeting between a doctor and a patient
- **Appointment Record**: The record created after an appointment, containing diagnosis and prescription details
- **Authentication**: The process of verifying user credentials (login validation)
- **Registration**: The process of creating a new user account in the system

- **Notification**: Automated messages (email or SMS) for reminders or confirmations
- **Report**: A document or summary generated by the doctor or receptionist showing appointments or medical data.